

Búsqueda informada y heurísticas

De costo uniforme a A^* : diseño, garantías y evaluación

Curso de Inteligencia Artificial
Semana 3 · Clase 3

Propósito de la clase

Incorporar conocimiento del problema para orientar la búsqueda sin confundir rapidez con garantía.

Al finalizar, podremos

- ▶ definir y auditar una función heurística;
- ▶ ejecutar búsqueda voraz y A^* ;
- ▶ distinguir admisibilidad de consistencia;
- ▶ explicar cuándo A^* requiere reaperturas;
- ▶ diseñar una comparación reproducible entre UCS y A^* .

Punto de partida

La clase anterior dejó establecidos:

- ▶ el problema $P = (S, A, T, s_0, G, c)$;
- ▶ nodo, frontera y conjunto explorado;
- ▶ control de ciclos y mejores costos;
- ▶ BFS, DFS y costo uniforme;
- ▶ garantías condicionadas por representación y costos.

Continuidad

UCS ordena por costo recorrido; ahora agregaremos una estimación del costo restante.

Pregunta de apertura

Pregunta clave

Si dos caminos cuestan lo mismo hasta ahora, ¿cómo decidir cuál explorar primero?

- ▶ ¿Qué conocimiento adicional está permitido?
- ▶ ¿En qué unidades debe expresarse?
- ▶ ¿Cómo sabemos si orienta sin engañar?
- ▶ ¿Qué garantía queremos conservar?

El esquema de búsqueda no cambia

```
insertar nodo inicial en frontera
mientras frontera no esté vacía:
    extraer el nodo con menor valor de prioridad
    descartar entradas obsoletas
    si es objetivo: devolver camino
    relajar cada transición legal
devolver fracaso
```

La estrategia cambia al definir la prioridad de la frontera.

Una familia, distintas prioridades

Estrategia	Prioridad	Pregunta que privilegia
BFS	profundidad	¿Cuántos pasos llevo?
UCS	$g(n)$	¿Cuánto costó llegar?
Voraz	$h(n)$	¿Qué parece más cerca?
A*	$g(n) + h(n)$	¿Qué solución total parece más barata?

La prioridad no modifica el grafo ni el costo real del camino.

Búsqueda no informada e informada

Ambas parten de $P = (S, A, T, s_0, G, c)$. La diferencia es la información usada para ordenar la frontera.

No informada: sin h

- ▶ Usa la definición del problema y el costo recorrido.
- ▶ No estima el costo que falta.
- ▶ Incluye BFS, DFS, limitada, iterativa y UCS.
- ▶ Prioriza por profundidad, orden o $g(n)$.

UCS pertenece aquí: $g(n)$ es costo exacto del pasado, no una estimación del futuro.

Informada: incorpora h

- ▶ Añade una estimación dirigida al objetivo.
- ▶ Voraz prioriza $h(n)$.
- ▶ A* prioriza $g(n) + h(n)$.
- ▶ La garantía depende de la estrategia y de h .

Voraz es informada, pero no es óptima en general.

No informada no significa aleatoria; informada no significa conocer el futuro ni garantizar optimalidad.

Punto de control: costo uniforme

UCS extrae el nodo con menor $g(n)$.

- ▶ Usa una cola de prioridad.
- ▶ Conserva el mejor costo conocido por estado.
- ▶ Prueba el objetivo al extraer, no al generar.
- ▶ Descarta entradas obsoletas.
- ▶ Es óptima bajo los supuestos declarados.

Límite

Puede expandir muchas alternativas baratas que se alejan del objetivo.

¿Qué añade la búsqueda informada?

Añade conocimiento para ordenar la exploración:

$$h(n) \approx \text{costo mínimo que falta desde } n \text{ hasta un objetivo}$$

- ▶ No altera $g(n)$.
- ▶ No crea conexiones inexistentes.
- ▶ No corrige estados insuficientes.
- ▶ No reemplaza restricciones.

Una heurística guía la búsqueda dentro del problema ya formulado.

Función heurística

Una heurística es una función:

$$h : S \rightarrow \mathbb{R}$$

- ▶ $h(n)$: estimación del costo restante;
- ▶ $h^*(n)$: costo óptimo restante verdadero;
- ▶ $h(g) = 0$ para todo estado objetivo g .

Convención

La heurística se evalúa sobre estados, aunque se escriba $h(n)$ por conveniencia.

Tres cantidades diferentes

Cantidad	Significado	Disponibilidad
$g(n)$	costo acumulado desde s_0	conocido al generar
$h(n)$	estimación del costo restante	calculado por la heurística
$f(n)$	prioridad de la estrategia	depende del algoritmo

En A*:

$$f(n) = g(n) + h(n)$$

f estima el costo total de una solución que pasa por n .

La unidad debe coincidir

Si g está en minutos, h también debe estar en minutos.

Costo objetivo	Heurística candidata
distancia	separación geométrica inferior
tiempo	distancia / velocidad máxima válida
cantidad de pasos	mínimo de pasos pendientes

Inválido

Sumar kilómetros y minutos sin una conversión justificada.

Dos extremos útiles

Heurística nula

$$h(n) = 0$$

A* se convierte en UCS. Es segura, pero no aporta orientación.

Heurística perfecta

$$h(n) = h^*(n)$$

Orienta idealmente, pero calcularla suele equivaler a resolver el problema.

Buscamos una aproximación barata, informativa y justificable.

Informativa no significa segura

Dimensión	Pregunta
Calidad empírica	¿Reduce expansiones y discrimina alternativas?
Garantía teórica	¿Respeto una relación demostrable con h^* y los costos?

No alcanza con parecer correcta

Una alta correlación con el costo verdadero no demuestra admisibilidad.

Lista de control inicial

Antes de usar h , declarar:

- 1 qué costo estima y en qué unidad;
- 2 qué información utiliza;
- 3 por qué vale cero en el objetivo;
- 4 si puede ser negativa;
- 5 cómo se comprobará consistencia;
- 6 cuánto cuesta calcularla.

Criterio

Una fórmula intuitiva no es todavía una heurística defendible.

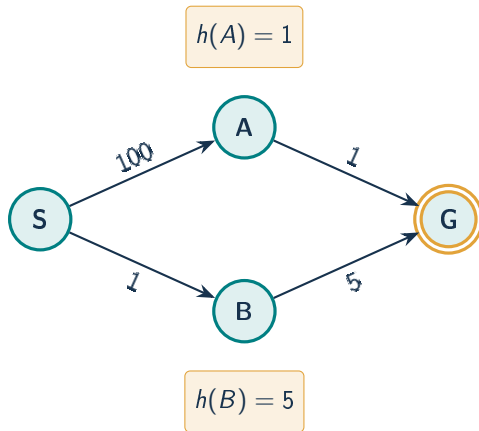
Búsqueda voraz por el mejor primero

La búsqueda voraz ordena la frontera solo por:

$$f(n) = h(n)$$

- ▶ Elige el estado que parece más cercano al objetivo.
- ▶ Ignora cuánto costó llegar hasta allí.
- ▶ Puede encontrar rápidamente una solución plausible.
- ▶ No garantiza el menor costo.

Contraejemplo para la búsqueda voraz



Voraz elige A y devuelve costo 101; existe $S \rightarrow B \rightarrow G$ de costo 6.

¿Por qué falla la voraz?

Nodo	g	h	prioridad voraz
A	100	1	1
B	1	5	5

Lectura

La heurística describe costo restante, no costo total.

Al ignorar g , una promesa cercana puede ocultar un pasado extremadamente caro.

Alcance de la búsqueda voraz

- ▶ En un grafo finito con control de repetidos, termina.
- ▶ En espacios infinitos no es completa en general.
- ▶ No es óptima, incluso si h es admisible.
- ▶ Debe evaluarse tanto velocidad como calidad de solución.

Uso razonable

Obtener una solución rápida cuando la optimalidad no es obligatoria.

Algoritmo A*

A* equilibra lo recorrido y lo que falta:

$$f(n) = g(n) + h(n)$$

- ▶ $g(n)$ aporta evidencia del camino ya recorrido.
- ▶ $h(n)$ orienta hacia regiones prometedoras.
- ▶ Se extrae el nodo con menor f .

Caso límite

Con $h = 0$, A* reproduce el comportamiento de UCS.

Qué debe guardar A*

Cada nodo conserva:

$$n = (\text{estado}, \text{padre}, \text{acción}, g, h, f)$$

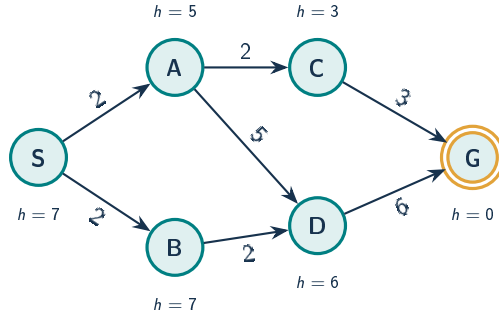
La implementación también necesita:

- ▶ cola de prioridad por f ;
- ▶ `mejor_g[estado]`;
- ▶ regla de desempate estable;
- ▶ padres para reconstruir el camino;
- ▶ política explícita de reapertura.

Pseudocódigo de A*

```
frontera <- prioridad por f
mejor_g[inicial] <- 0
insertar inicial con f = h(inicial)
mientras haya frontera:
    n <- extraer mínimo
    si n.g es obsoleto: continuar
    si n es objetivo: devolver camino
    para cada sucesor:
        nuevo_g <- n.g + costo
        si nuevo_g mejora mejor_g:
            registrar e insertar con f = nuevo_g + h
devolver fracaso
```

Grafo didáctico para A*



Aristas dirigidas · costos no negativos · desempate por orden de inserción.

A*: inicialización

Nodo	g	h	f
S	0	7	7

Primera extracción

Se extrae S porque es el único nodo disponible.

Regla de desempate del ejemplo: **orden de inserción**.

A*: después de expandir S

Frontera	g	h	f
A	2	5	7
B	2	7	9

Decisión

A* extrae A. Ambos caminos recorridos cuestan lo mismo; la heurística rompe la igualdad.

A*: de A al objetivo

Frontera tras expandir A	g	h	f
C	4	3	7
B	2	7	9
D	7	6	13

Al expandir C aparece G con $g = f = 7$. Al extraer G :

$$S \rightarrow A \rightarrow C \rightarrow G, \quad C^* = 7$$

UCS y A* sobre la misma instancia

Resultado	UCS	A*
Camino	$S - A - C - G$	$S - A - C - G$
Costo	7	7
Prioridad	g	$g + h$
Expandidos antes de extraer G	S, A, B, C, D	S, A, C

La heurística reduce expansiones en este ejemplo; no reduce el costo óptimo.

A* no significa óptimo automáticamente

La garantía depende de:

- ▶ ramificación finita;
- ▶ costos no negativos y, para terminación, $c \geq \varepsilon > 0$;
- ▶ objetivo comprobado al extraer;
- ▶ conservación del mejor g ;
- ▶ descarte de entradas obsoletas;
- ▶ reapertura cuando sea necesaria;
- ▶ propiedades demostradas de h ;
- ▶ unidades compatibles.

El nombre del algoritmo no sustituye estas condiciones.

Heurística admisible

Una heurística es admisible si nunca sobreestima:

$$0 \leq h(n) \leq h^*(n)$$

- ▶ Es una cota inferior, no una predicción promedio.
- ▶ Debe cumplirse para todos los estados relevantes.
- ▶ Una sola sobreestimación rompe la garantía general.

Alcance

Admisibilidad se refiere al costo óptimo restante global.

Intuición de optimalidad

Para un nodo sobre un camino óptimo:

$$f(n) = g(n) + h(n) \leq C^*$$

Para una solución subóptima de costo $C > C^*$:

$$f(G) = g(G) = C > C^*$$

Lectura

La frontera conserva antes algún candidato compatible con costo óptimo, siempre que se respeten las demás condiciones de A^* .

Heurística consistente

Para toda transición $n \rightarrow n'$:

$$h(n) \leq c(n, n') + h(n')$$

Es una desigualdad triangular. Entonces:

$$f(n') = g(n) + c(n, n') + h(n') \geq g(n) + h(n) = f(n)$$

Consecuencia

Con $h(G) = 0$ y $h(n) \geq 0$, consistencia implica la admisibilidad adoptada y los valores f no disminuyen.

Auditoría local de consistencia

Arista	Comprobación	Arista	Comprobación
$S \rightarrow A$	$7 \leq 2 + 5$	$S \rightarrow B$	$7 \leq 2 + 7$
$A \rightarrow C$	$5 \leq 2 + 3$	$A \rightarrow D$	$5 \leq 5 + 6$
$B \rightarrow D$	$7 \leq 2 + 6$	$C \rightarrow G$	$3 \leq 3 + 0$
$D \rightarrow G$	$6 \leq 6 + 0$		

Cero violaciones demuestra consistencia **en este grafo revisado**, no en datos futuros.

Admisible pero inconsistente

Admisibilidad no garantiza consistencia.

- ▶ Los valores f pueden disminuir a lo largo de un camino.
- ▶ Puede aparecer después un camino más barato hacia un estado cerrado.
- ▶ A* debe permitir la **reapertura** cuando mejora $mejor_g$.
- ▶ Cerrar estados para siempre puede producir una solución subóptima.

Regla de implementación

No prohibir reaperturas sin haber demostrado consistencia.

Cómo diseñar una heurística

Principio	Idea
Relajación	eliminar restricciones y resolver el problema más fácil
Abstracción	buscar exactamente en un espacio reducido
Geometría	usar distancias y límites físicos
Descomposición	combinar subcostos sin doble conteo
Patrones	precomputar costos de configuraciones abstractas

La procedencia de la cota debe quedar documentada.

Combinar y evaluar heurísticas

Si h_1 y h_2 son admisibles:

- ▶ $\max(h_1, h_2)$ también es admisible y domina a ambas;
- ▶ $h_1 + h_2$ requiere costos aditivos sin superposición.

Evaluar sobre las mismas instancias

- ▶ costo de solución;
- ▶ nodos generados y expandidos;
- ▶ frontera máxima y reaperturas;
- ▶ tiempo de búsqueda y costo de calcular h .

Comparación de estrategias

Estrategia	Prioridad	Óptima	Riesgo principal
BFS	profundidad	costos iguales*	memoria
UCS	g	sí*	expansión amplia
Voraz	h	no	solución cara
A*	$g + h$	sí*	memoria y supuestos

* Bajo ramificación finita, costo mínimo positivo cuando corresponda, prueba correcta del objetivo, repetidos y propiedad heurística declarada.

MOV-02: movilidad como grafo OD

Esta es una abstracción distinta de la reasignación de flota: se buscan caminos sobre conectividad OD agregada.

Representación

- ▶ Nodo: zona TLC.
- ▶ Arista $i \rightarrow j$: suficientes viajes reportados.
- ▶ Costo: duración o distancia mediana.
- ▶ Referencia: UCS sobre la misma instancia.

Heurística candidata

Separación geográfica con escala y unidad compatibles, auditada arista por arista.

Límite

El camino no es una ruta vial ni una trayectoria observada completa.

Actividad: definir y probar una heurística

En equipos:

- 1 fijar origen, objetivo, costos y desempate;
- 2 ejecutar UCS como referencia;
- 3 proponer h en la misma unidad;
- 4 comprobar $h(G) = 0$, no negatividad y consistencia;
- 5 ejecutar A^* bajo idénticas condiciones;
- 6 comparar costo, camino, generados, expandidos y frontera máxima;
- 7 explicar qué ocurre con $h = 0$.

Producto

Definición y prueba de la heurística, sin resultados inventados.

Ideas centrales

- ▶ Voraz usa h ; A^* combina $g + h$.
- ▶ Admisibilidad es global; consistencia se audita localmente.
- ▶ A^* necesita implementación y supuestos correctos.
- ▶ Menos expansiones no significa menor costo.
- ▶ h comparte unidad y semántica con el costo.

Lectura

Capítulo 7, §7.3

Capítulo 6, §6.4

Actividad MOV-02

Una heurística útil también debe ser auditable.